

Changes from R0

Incorporated the suggestions made by CWG in Kona/Toronto.

- Enable brace-elision by having [dcl.init.list] deduce the type off a declaration with a corresponding initializer.
- Instead of creating a new paragraph 13.3.3.2, add a bullet to (3.1).
- The rules were adjusted to match better the one introduced by core issue 1307; the arrays now need to have the same element type for the tie-breaker in [over.ics.rank] to apply.
- Allow more general pointer conversions by incorporating the bound conversion into qualification conversions.

Motivation and impact

As of core issue 393, function parameters can be pointers or references to arrays of unknown bound. However, binding such a parameter to an array of known bound isn't permitted:

```
void f(int(&)[]);
int arr[1];

f(arr);           // Error
int(&r)[] = arr;  // Error
```

This restriction is unjustified and should be removed. This might break code, since overload resolution will potentially select different functions when there are corresponding overloads. However, note that such parameters were only allowed a while ago, with GCC not yet supporting them. The practical impact is therefore presumably small.

Approach and decisions

The initialization of pointers to arrays of unknown bound will be allowed by extending qualification conversions to drop bounds. Reference initialization rules will be adjusted by modifying reference compability. Such bindings have Exact Match rank. We also propose to allow list-initialization for references to arrays of unknown bound by deducing the array temporary's size.

Ranking of reference initialization conversions

Consider

```
void f(int(&)[ ]),      // (1)
      f(int(&)[1]),     // (2)
      f(int*);         // (3)

void h(int(*)[ ]),     // (a)
      h(int(*)[1]);    // (b)
```

(2) and (b) should clearly be better than (1) and (a), respectively, as the former overloads are more restricted.

Furthermore, (3) should be equal to (1) (as it is to (2)). To maintain this ambiguity, (1) must be given Exact Match rank. We do not favor this ordering and feel that (3) is conceptually less specialized than (2) and even (1); however, consistency is more important than fixing part of the problem for arrays of unknown bounds.

Finally, (a) should be worse than (b), which is implied by (a) necessitating a qualification conversion in that case.

Ranking of list-initialization sequences

We also propose to allow list-initialization and introduce corresponding rules in overload resolution:

```
int b(int (&&)[ ] );    // #1
int b(long (&&)[ ] );   // #2

int b(int (&&)[1]);     // #3
int b(long (&&)[1]);    // #4

int b(int (&&)[2]);     // #5

b({1});
```

Here,

- #1, #3 and #5 should rank better than both #2 and #4, as no promotion is necessitated. This is consistent with the approach of core issue 1307, [CWG1307](#) whose resolution this paper extends.
- #1 should rank worse than #3, being far less specialized.
- #1 should rank better than #5, as the latter requires a larger array temporary.
(#3 also ranks better than #5 for the same reason--cf. core issue 1307).

For these reasons, only if the arrays are of the same type, the size of the referenced array and the known/unknown are primary criterions, (unconditionally) followed by the worst performed conversion.

Proposed wording

This is based on N4659.

☐ Hide deleted wording

Modify 7.5 [conv.qual] as follows, where 8 [expr] ¶14 (which defines cv-combined types) has been moved to the beginning of the third paragraph:

1. A *cv-decomposition* of a type T is a sequence of cv_i and P_i such that T is

$$“cv_0 P_0 cv_1 P_1 \dots cv_{n-1} P_{n-1} cv_n U” \text{ for } n > 0,$$

where each cv_i is a set of cv-qualifiers (6.9.3), and each P_i is “pointer to” (11.3.1), “pointer to member of class C_i of type” (11.3.3), “array of N_i ”, or “array of unknown bound of” (11.3.4). If P_i designates an array, the cv-qualifiers cv_{i+1} on the element type are also taken as the cv-qualifiers cv_i of the array. [*Example*: The type denoted by the *type-id* `const int **` has two cv-decompositions, taking U as “int” and as “pointer to `const int`”. *end example*] The n -tuple of cv-qualifiers after the first one in the longest cv-decomposition of T , that is, $cv_1, cv_2, \dots, cv_n$, is called the cv-qualification signature of T .

2. Two types T_1 and T_2 are *similar* if they have cv-decompositions with the same n such that corresponding P_i components are **either the same or one is “array of N_i ” and the other is “array of unknown bound of”**, and the types denoted by U are the same.
3. The *cv-combined type* of two types T_1 and T_2 is **a** the type T_3 similar to T_1 whose **cv-qualification signature** **cv-decomposition** (7.5) is:
- for every $i > 0$,
 - cv_i^3 is the union of cv_i^1 and cv_i^2 ;
 - **if either P_i^1 or P_i^2 is “array of unknown bound of”, P_i^3 is “array of unknown bound of”; otherwise it is P_i^1 .**
 - if the resulting cv_i^3 is different from cv_i^1 or cv_i^2 , **or the resulting P_i^3 is different from P_i^1 or P_i^2** , then `const` is added to every cv_k^3 for $0 < k < i$.

[*Note*: Given similar types T_1 and T_2 , this construction ensures that both can be converted to T_3 . *end note*] A prvalue expression of type T_1 can be converted to type T_2 if **the cv-combined type (clause 8) of T_1 and T_2 is T_2 . the following conditions are satisfied, where cv_i^j denotes the cv-qualifiers in the cv-qualification signature of T_j :**

(3.1) — T_1 and T_2 are similar.

(3.2) — For every $i > 0$, if `const` is in cv_i^1 then `const` is in cv_i^2 , and similarly for `volatile`.

(3.3) — If the cv_i^1 and cv_i^2 are different, then `const` is in every cv_k^1 for $0 < k < i$.

Drafting note: it may be worth to consider renaming certain terms starting with “cv”, e.g. cv-combined types are not merely combining cv-qualifiers anymore. Possibly an editorial issue?

Delete 8 [expr] ¶14.

Modify 8.2.11 [expr.const.cast] ¶3:

For two similar types T_1 and T_2 , a prvalue of type T_1 may be explicitly converted to the type T_2 using a `const_cast` **if, considering the cv-decompositions (7.5) of both types, all P_i^1 are the same as P_i^2 .**

Modify 11.6.3 [dcl.init.ref] ¶4 as follows:

Given types “ $cv_1 T_1$ ” and “ $cv_2 T_2$ ”, “ $cv_1 T_1$ ” is reference-related to “ $cv_2 T_2$ ” if T_1 is the same type as T_2 , **or T_1 is a base class of T_2 , or T_1 is an array type of unknown bound of U and T_2 is an array type of known bound of U for some type U .**

Modify 11.6.4 [dcl.init.list] ¶(3.8) as follows:

Otherwise, if T is a reference type, a prvalue ~~of the type referenced by T~~ is generated. The prvalue initializes its result object by copy-list-initialization or direct-list-initialization, depending on the kind of initialization for the reference. The prvalue is then used to direct-initialize the reference. **The type of the temporary is the type referenced by T, unless T is “reference to array of unknown bound of U”, in which case the type of the temporary is the type of x in the declaration U x[] braced-init-list, where braced-init-list is the initializer list.**

Modify 16.3.3.1.5 [over.ics.list] ¶5 as follows:

Otherwise, if the parameter type is “array of N X” **or “array of unknown bound of X”**, if there exists an implicit conversion sequence ~~for each element of the array from the corresponding~~ **from each** element of the initializer list (~~or~~ **and** from { } if ~~there is no such element~~ **N exceeds the number of elements in the initializer list**) **to X**, the implicit conversion sequence is the worst such implicit conversion sequence.

Augment 16.3.3.2 [over.ics.rank] ¶(3.1) as indicated:

List-initialization sequence L1 is a better conversion sequence than list-initialization sequence L2 if

- L1 converts to `std::initializer_list<X>` for some X and L2 does not, or, if not that,
- L1 converts to type “array of N1 T”, L2 converts to type “array of N2 T”, and N1 is smaller than N2, **or if not that,**
- **L1 converts to type “array of unknown bound of X” and L2 converts to type “array of N X”, where a variable of the former type, if initialized with the initializer list, would have an array type with a different bound than N, or if not that,**
- **L1 converts to an array of known bound of X and L2 converts to an array of unknown bound of X,**

even if one of the other rules in this paragraph would otherwise apply. **[Example:**

```
void f(int    (&&)[ ] );    // #1
void f(double (&&)[ ] );    // #2
void f(int    (&&)[2] );    // #3

f( {1} );                // Calls #1: Better than #2 due to conversion, better than #3 due to bounds
f( {1.0} );              // Calls #2: Identity conversion is better than floating-integral conversion
f( {1.0, 2.0} );         // Calls #2: Identity conversion is better than floating-integral conversion
f( {1, 2} );              // Calls #3: Converting to array of known bound is better than to unknown bound,
                          //                and an identity conversion is better than floating-integral conversion
```

— end example] [Example:...— end example]

Modify 16.3.3.2 [over.ics.rank] ¶(3.2.5):

S1 and S2 differ only in their qualification conversion and yield similar types T1 and T2 (7.5), respectively, ~~and the cv-qualification signature of type T1 is a proper subset of the cv-qualification signature of type T2~~ **where T1 can be converted to T2 by a qualification conversion (7.5).**

Acknowledgments

The author would like to thank David Krauss and Johannes Schaub for their valuable feedback.

References

[EWG118] “[tiny] Allow conversion from pointer to array of known bound to pointer to array of unknown bound”: wg21.link/ewg118

[CWG393] “Pointer to array of unknown bound in template argument list in parameter”: wg21.link/cwg393

[CWG1307] “Overload resolution based on size of array *initializer-list*”: wg21.link/cwg1307